

PROCESSES AND THREADS

This chapter introduces some fundamental concepts of operating systems. Since this is the introductory chapter of the course, we will use rather vague wording in numerous places. In later chapters many of these descriptions will be made more concrete.

Computer architecture in a nutshell

A computer system logically consists of four components: the processor, memory, input/output (I/O) devices, and a connection element that allows information to be exchanged between the first three components. The processor is the entity that will effectively execute the instructions. These instructions, as well as the data that is manipulated by them, are (mostly) in memory. The I/O devices (e.g. hard disk, graphics card, network card, keyboard, audio devices) allow information to be exchanged with the system.

Memory is a collection of bits divided into words of fixed length (e.g. 16, 32 or 64 bits). These words are numbered, starting from zero, where we refer to the word number by speaking of the address of a memory word. Each of these memory words can contain data or an instruction. When it is an instruction, the first bits, which we call the *opcode*, indicate the instruction, while the remaining bits contain a memory address. For example, when a data word is to be read from memory, the opcode will indicate that it is a read operation, while the address indicates which word is to be read.

The processor's goal is to sequentially execute the instructions that make up a program. To accomplish this, the processor has, among other things, a limited number of registers (which are much smaller in number than the number of memory words). The processor must know which instruction is to be executed, this is always stored in the *instruction* register. The memory address of the next instruction is held by the *program counter*, whose value is also held in a register.

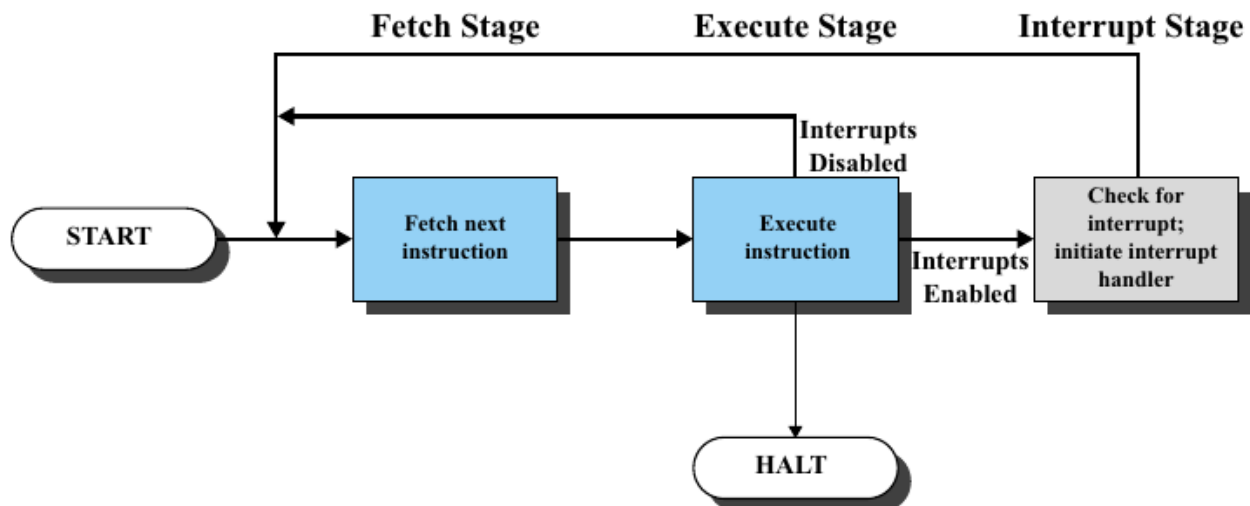


Fig. 1: Instructional cycle with interrupts [Stallings 03].

Logically, the processor runs through a cycle where (a) each time an instruction is performed and (b) the processor calls the next instruction from memory. The program counter is increased by one each time the instruction is executed. This means that a large portion of the instructions are in sequence in memory. In addition, there may also be *branch* instructions that set the value of the program counter equal to the supplied value (to handle function calls, among other things).

To properly use the capacity of the processor, it should be ensured that the processor does not have to wait too long between the execution of two instructions. However, when an operation involves

an I/O device, it will typically take thousands of cycles for the I/O device to process the processor's request. This causes the operating system to temporarily halt the current process so that another process can make useful use of the processor. To indicate that the I/O operation is finished, the *interrupt* mechanism will be used.

Logically, the cycle of the processor will then consist of three phases (see also Figure 1): (a) execute an instruction, (b) check whether an interrupt has taken place and possibly process it and (c) request the next instruction. This interrupt mechanism is not only used for I/O operations, but for all *system calls*, *exceptions* and *device interrupts*. With the help of a system call a user program can ask the operating system for a certain service, like opening a file. Exceptions are illegal operations, such as dividing by zero, or requesting an invalid memory address. Device interrupts are used to indicate that a certain hardware device needs attention from the processor. This is for example the case when data is read from the hard disk or network card, but it can also come from the so called *clock chip* that generates a clock interrupt several times a second to intertwine the execution of different processes. When the interrupt comes from a user program because of a system call or exception that has occurred, one also speaks of a *trap*.

The code to handle an interrupt, that is the *interrupt handler*, as well as the code used to determine which interrupts are forwarded to which processor are part of the operating system.

1 Processes

Roughly speaking, we can say that a process corresponds to a program in progress. The task of the operating system is therefore to manage multiple processes. For example, it must ensure that the processor capacity is properly utilized. The operating system plays a decisive role in this as it will decide the order in which the processes can access the processor. We will return to this issue in the *Scheduling* section. Furthermore, the processes must share all available resources, especially memory. The operating system will distribute the available resources and must resolve any resulting conflicts. We will discuss this in more detail in the chapters on Memory Management and Concurrency.

1.1 Process attributes

First, let's consider the physical manifestation of a process. First, a process consists of a set of instructions, or still, the code of the process. Then there is the data that is manipulated by the code. The extent of this data may be fixed already at compilation or may change dynamically over time. In the first case we speak of static data, otherwise of dynamic data.

The data and code is loaded into the (secondary) memory when a process is started (for more info we refer to the section Memory Management). Furthermore, each process will use a *user stack* and a *heap* part. The user stack is used to handle procedure calls and is organized in the form of a stack structure, while memory allocated during the execution of a process is added to the heap portion.

Suppose we call a procedure during the execution of a process. Then the program counter will be adjusted so that it points to the beginning of the procedure code. When the procedure ends, the processor must return to the location from which the call took place. To determine the program counter, the return address must be stored. For this a stack structure is used. Namely, whenever a procedure call occurs, we place the return address at the top of the stack and when a procedure ends we return to the address at the top of the stack and then remove this address. In this way we can handle nested and recursive procedure calls.

Furthermore, one can also place on the stack the local variables (and parameters provided) of the procedure. This is very useful since one cannot foresee exactly how many procedure calls will take place. The stack also contains a frame pointer to allow a variable size (in number or size) of the

local parameters (see Figure 2). Return parameters are also passed through the stack. The address of the top of the stack is kept in one of the processor's registers. In short, the stack is a dynamic component of a process that makes efficient use of memory. Usually there is a maximum size for the user stack.

In addition to the code, data, the system stack and the heap part, each process must also maintain information that is used by the operating system to manage the processes. This information, the process control block, is often divided into three parts:

1. *Process Identification*: Each process is given a unique number at startup. This number is not only used for inter-process communication, but also as an index to the process tables of the operating system to quickly find information (e.g. for the system to quickly look up information (see below). When a process is created by another process (the *parent* process), its identifier is also kept its identifier is also kept. This is for example the case when we execute a command from the shell. In multi-user systems each process also has a user identifier.

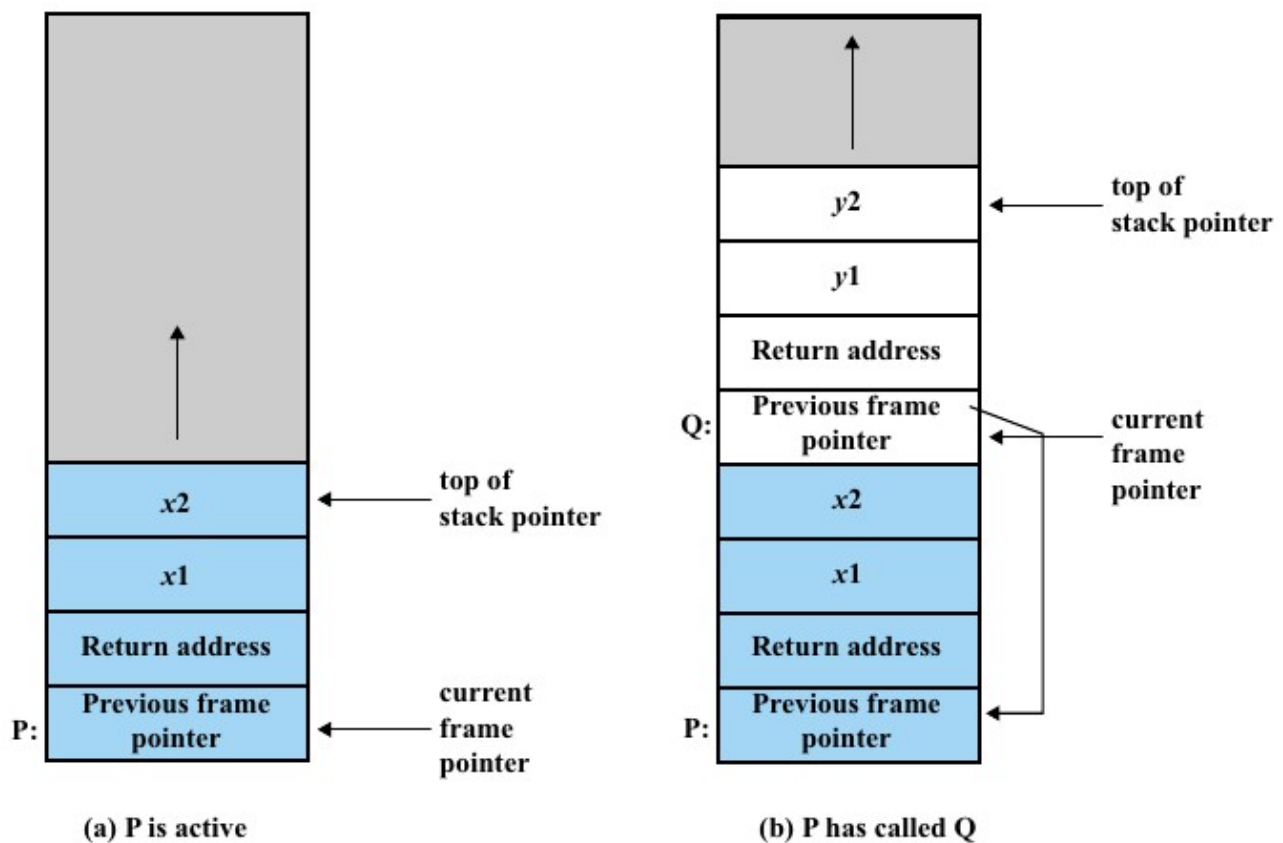


Fig. 2: Evolution of the system stack of a process [Stallings 03].

2. *Processor State Information*: This data reflects the contents of the processor registers. When the process is in execution, this info is in the processor itself. If the process is interrupted by another process, the content of the registers will be written here.

3. *Process Control Information*: All additional information that the operating system needs to manage the processes can be found here. Among other things, the state of the process is kept here and the scheduling information (see below), information regarding the memory that is available to the process, the privileges of the process, fields needed to build the process tables, etc.

The whole of the code, data, stack and process control block is called the *process image*.

The operating system will organize the process information contained in the control blocks of all processes into a number of tables. There are memory tables, I/O tables, file tables and a process table that is organized so that the process id can be used as an index to the table. The memory tables indicate for example where and in what form the process images of each of the processes are located in (virtual) memory, further on in the course we discuss some possible structures. I/O tables indicate which resources are in use and maintain information related to the data structures needed to support I/O management.

1.2 Process states

When the operating system manages multiple processes, it must keep track of which process is in execution, which are waiting for an I/O event, and which processes can continue execution (if the process in execution releases the processor). For this purpose, the *process state* is used.

The possible states differ from system to system. However, a number of states return, albeit under different names, in most systems. The process that has access to the processor is in the state *Running*. A process in progress can be interrupted for several reasons without waiting for an I/O event. For example, when an interrupt occurs due to a key being pressed on the keyboard. Furthermore, a process in most systems only a limited time uninterrupted access to the processor (one uses the term *time-sharing systems*). To achieve this, the clock chip will regularly (e.g. every 100ms) generate a clock interrupt, so that the process in execution can be interrupted. We come back to this matter in detail in the section Scheduling. When a process is interrupted by the operating system while it can actually continue to make useful use of the processor, the state of the process changes to *Ready*. The process is no longer in execution, but can continue to execute should the processor be free.

A process can also be interrupted because it must wait for an I/O event (or for another service offered by the operating system). Sometimes a process also has to wait for information from another process or for a shared resource to be freed (e.g., a portion of memory that is being manipulated by multiple processes). In all these cases, the process is interrupted and we set the state of the process to *Blocked*. Most systems further have a *New* and *Exit* state (see Figure 3).

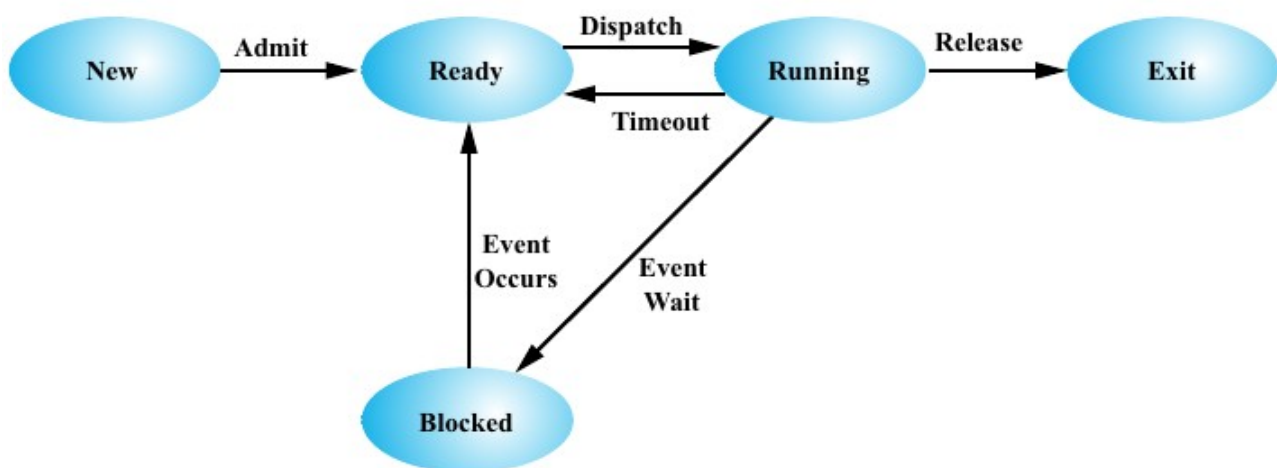


Fig. 3: Process model with five states [Stallings 03].

A process is in the *New* state just after it is created and remains in this state until it is ready for execution. The *Exit* state is reached when the process terminates (possibly due to the occurrence of an error). When a process is in the *Exit* state, the process is finished but data is still present that can be retrieved by the parent process (e.g. for debugging).

Another commonly used state is the *Suspend* state. The process image of a process that enters this state will be written to secondary memory (typically the swap space of a disk). To make good use

of the processor's capacity, there must often be many active processes. However, these all require memory, which is a rather scarce resource. Although the process image does not have to be completely present in memory (see Virtual Memory section), it is often necessary to temporarily write a number of process images (preferably those in a *Blocked* state) to disk. There are also other reasons to temporarily remove a process from memory. For example, a process may be run periodically and go into the Suspend state during the intervals. In some cases, a Ready process can also be referred to swap space (mainly when another process requires a very large block of memory). Hence, the split *Ready*, suspend and *Blocked*, *suspend* can also be useful.

1.3 Kernel, state and mode switching

The kernel is formed by the main functions of the operating system. One speaks of a *monolithic* kernel if the entire operating system corresponds to the kernel (as, for example, in Linux), otherwise one speaks of a *microkernel*. To protect the system from programming errors, the processor will use at least 2 privilege levels: user and kernel level (the x86 architecture supports 4 privilege levels) and not all instructions (and interrupts) may be executed in user mode. When a user process executes a system call, the processor will consequently switch to kernel mode first. This ensures, among other things, that all data that makes up the process control block cannot be modified by a user process.

We already indicated that a process can be interrupted because of an interrupt. When another process takes over the processor, the state information of the interrupted process must be removed from the registers (and stored in memory). Furthermore, the necessary fields in the process tables have to be modified for both the incoming and outgoing process. However, not every interrupt needs to trigger a *process switch*. For example, when the process calls a system call or a keyboard interrupt occurs, the ongoing process may continue as usual when the interrupt is resolved. In this case only the data from the registers must be saved and the other data in the process control block need not be modified (the process remains all the time in the *Running* state). In this case we speak of a *mode switch*, which requires less work than a process switch.

Even the applied *page table*, which we will discuss in detail later in the chapter Virtual Memory, remains unchanged with a mode switch. This is because the kernel functionality is part of every user process. Specifically, all kernel functionality will reside in the virtual address space of each process (typically in the top 2 Gbyte in a 32 bit machine). Thus, the system calls are executed as being part of the user process. Of course, it is sufficient to store a single copy of the code of these routines in memory (by configuring the page tables in this way). When a user process switches to kernel mode it will use a separate stack for possible function calls: the *kernel stack*. It also contains the information that was in the processor registers just before the mode switch. One of the reasons for having a separate kernel stack and not just using the user stack for these calls is that it can already be corrupted by programming errors in the user process code. When a process is in kernel mode, it can still be interrupted unless the so-called *interrupt flag* is set in the processor.

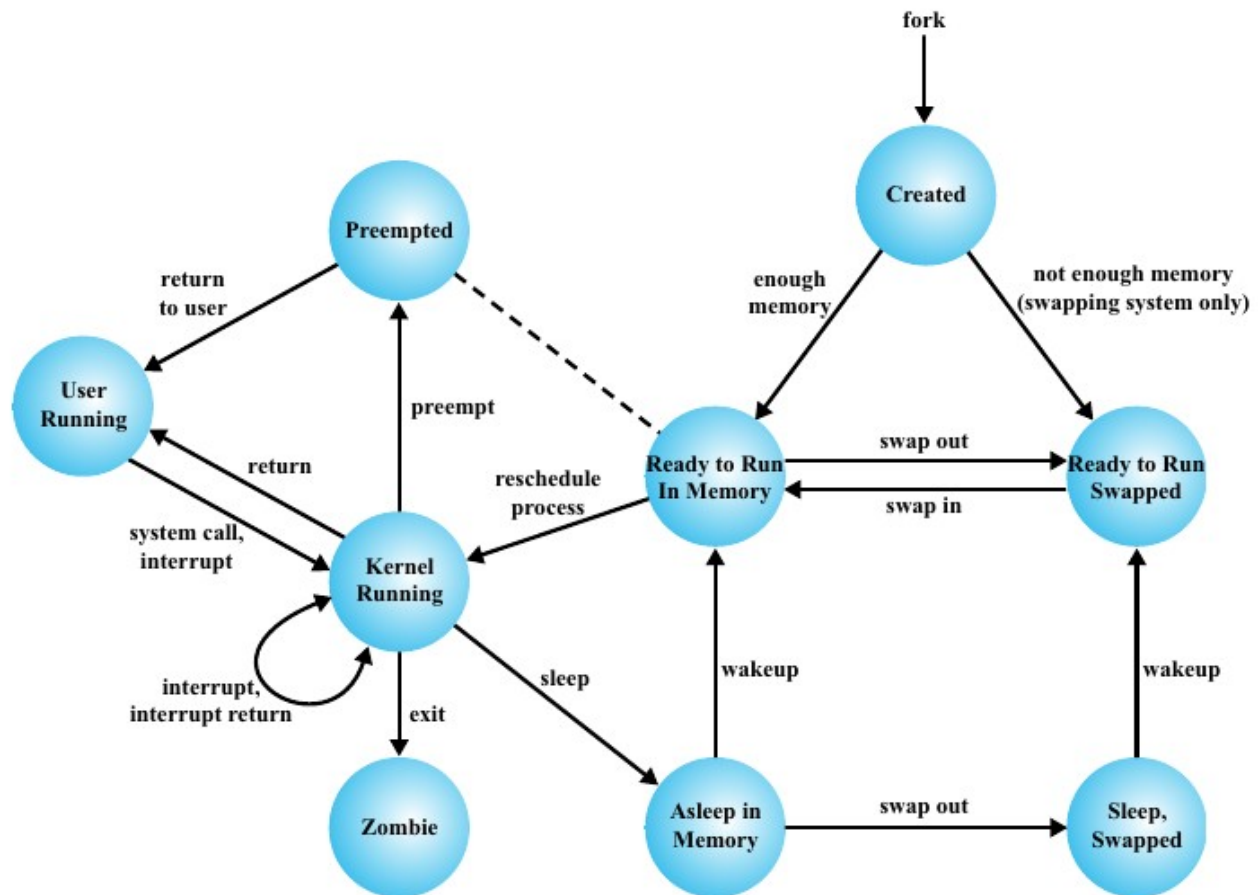


Fig. 4: UNIX state diagram [Stallings 03].

An example of a state diagram of a UNIX system is shown in Figure 4. In this system, the *Running* state is split into two states: *User Running* and *Kernel Running*. This indicates whether the process is in kernel or user mode. The *Ready* state is also split into the *Preempted* and *Ready to Run* states. When a process switches from kernel mode back to user mode, but the scheduler decides to let another process run, the interrupted process enters the *Preempted* state. This is for example the case when a clock interrupt occurs when a process is in user mode. In this case the process first enters kernel mode (because the scheduler is part of the kernel) and if it then decides to allow another process to run it enters *Preempted* state. The processes in the *Preempted* state are also *Ready to Run* and these two states together form one queue. Finally, the blocked state is split into *Sleep, Swapped* and *Asleep in Memory*. Note that a transition to these states is also done via the *Kernel Running* state.

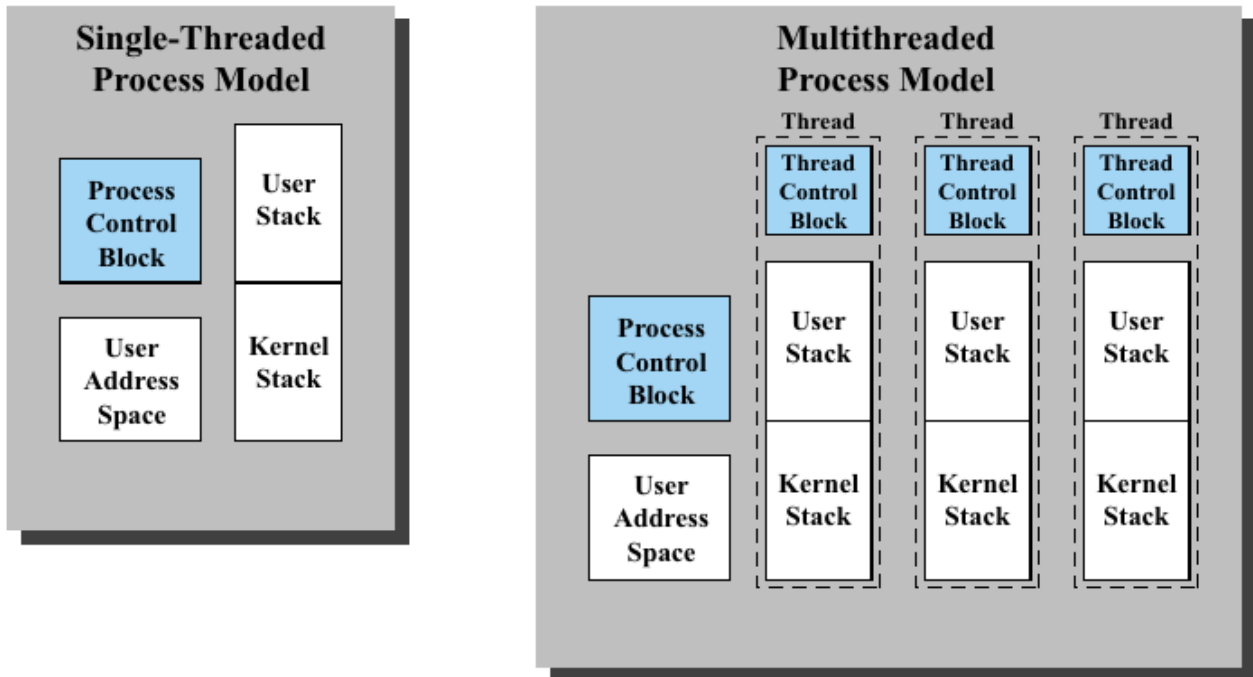


Fig. 5: Single and multithreaded processes [Stallings 03].

2 Threads

With a process we associate on the one hand the data related to the resources it has at its disposal, e.g. the (virtual) memory, I/O devices, files, etc. On the other hand, there is also the sequence of instructions that is executed and that may be interrupted several times. The state of the process also changes regularly when the instructions are executed.

Many programs could be divided into several processes with which the same resources are associated, but which each go through a different path of instructions. For example, in the case of a Web server, we could create a process per request so that we can request several files simultaneously (i.e., make the waiting of the I/O requests parallel). In the case of a word processor, we could provide an automatic save function in the form of a separate process that regularly saves the text to an I/O buffer. For a spreadsheet, one could run the redrawing of the screen and the processing of the user input as separate processes (to increase the responsiveness of the program).

These different processes then each have their own process image. The joint code of execution and even the data can be shared. However, for protection reasons, the processor will have to go into kernel mode to write into the shared memory areas. Furthermore, each process has its own process control block and multiple process switches must occur if we want to interleave process execution. In addition, the lifetime of these processes is sometimes very short, so the overhead for creating and undoing the process is very large. In short, none of this adds to the efficiency.

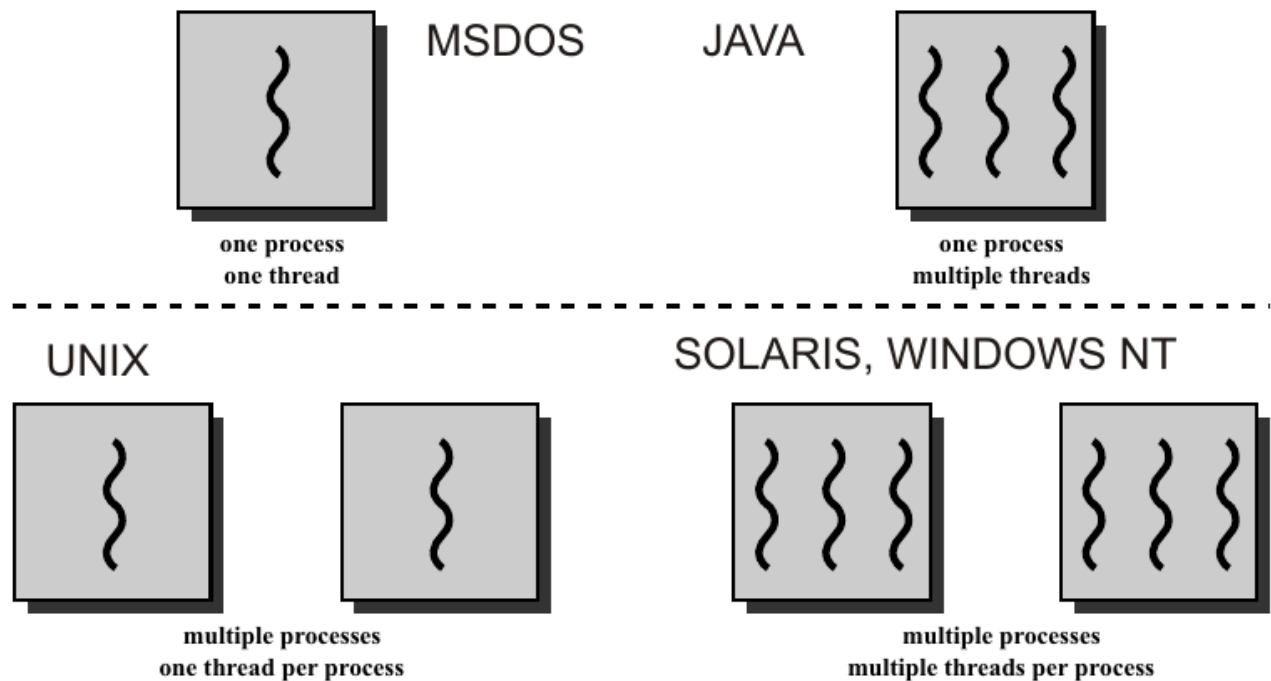


Fig. 6: Processes and Threads [Stallings 03].

To still be able to partition a program in a more performant way, one can use *threads*. Threads that are part of the same process will share the same resources (i.e., the code/data and the process control block), but can each follow a different path of execution. To realize this, they do have their own user (and kernel) stack, see Figure 5. Thus, a process consists of multiple threads that can each be intertwined. Hence, each thread also has its own state.

Possible states are *Running*, *Ready* and *Blocked*. The swapped state has little use, because a single thread is not swapped. Only processes are swapped and when this happens all corresponding threads will be inactive as well, since the code and data part needed for execution is on disk. Also when the process ends, all threads end. In the thread control block we find this state, together with other information to schedule and synchronize the threads. The content of registers (of the thread) must of course also be stored per thread, since threads can interrupt each other. As with the shared resources between processes, the common data of the threads may need to be protected from simultaneous access (see chapter Concurrency).

When a thread wants to exchange data with other threads, it only needs to write the info to the data area. This makes the info immediately available to all other processes. If another thread later becomes active and reads the info, we have exchanged info without a process change or even a mode change. Also when a thread opens a certain file (or another I/O resource), it is immediately available to all other threads belonging to the process.

2.1 User-level and kernel-level threads

Threads can be implemented roughly in two ways: user-level threads (ULTs) and kernel-level threads (KLTs). In the case of ULTs, the entire thread management is performed by the process to which the threads belong. More so, the kernel does not even know that the process consists of multiple threads.

In this case, the necessary software to create and manage threads is provided by means of a *thread library*, which is a collection of routines to create, terminate, schedule threads or to exchange messages between threads (e.g., the Java virtual machine).

When starting a process, one thread is usually created and it will, through the library routines, create the other threads. When the kernel decides to interrupt the process in progress, e.g., because of an I/O interrupt or a clock interrupt, the kernel will claim the processor and possibly later allow this or another process to continue. During this entire time, the state of the thread that was in execution will remain the same as *Running*. So looking at the thread states, it seems that the other processes do not exist and we are only executing these threads on a *dedicated* machine. Note that a process can even be interrupted during a thread switch.

ULTs have a number of advantages:

1. Thread swaps happen in the context of a process, without involving the kernel, which is very efficient.
2. The scheduling of threads is not being influenced by the kernel. An appropriate scheduling strategy can therefore be designed for each application.
3. ULTs are processor independent and do not require kernel changes.

There are, of course, some limitations:

1. When a thread calls a kernel function, the kernel will in many cases place the process that made the call in the *Blocked* state until the corresponding event occurs. This causes all threads to be blocked at once.
2. When the system has multiple processors, only one thread will be running.

The problem of blocking can partly be solved by using *jacketing*. An I/O jacket routine of an application will convert the blocking call into a non-blocking call by first checking if the I/O device is available. If this is not the case, the thread will, via the thread library, pass control to another thread and set its state to *Ready* (and not *Blocked*).

Kernel-level threads are visible to the kernel, moreover, the kernel will take care of thread management. All scheduling decisions will be made at thread level (instead of process level). This approach obviously solves both drawbacks of ULTs, but it has its own limitations.

The main limitation is efficiency: a single thread change requires kernel intervention and a mode switch is thus necessary every time. Experiments have shown that the overhead for creating, scheduling, executing and terminating KLTs is 10 to 30 times slower than ULTs. However, the same operations at the process level are still up to 10 times slower. The choice between ULTs and KLTs is often very application dependent. If most thread switches require kernel interference anyway then the performance advantage of ULTs is quickly lost.